

# Linux Cheat Sheet

©Maximilien Notz 2026

## Linux File Structure

| | |
|---|---|
| `/boot` | Boot loader files |
| `/bin` | executable binaries |
| `/home` | Users home directory |
| `/etc` | system config files |
| `/var/log` | log files |
| `/usr/local` | software manualy compiled or installed manually |
| `/dev/null` | when data is sent to this file, the data disappears. |

### Path shortcute

| | |
|---|---|
| `~` | home directory |
| `/` | root directory |
| `.` | current directory |
| `..` | parent directory |
| `-` | previous directory |

## Linux Terminal

### General Commands

| | |
|---|---|
| `clear` | Clear the terminal screen. |
| `history` | Display the command history. |
| `cal` | Display a calendar. |
| `uptime` | Display how long the system has been running. |
| `man` | Display the manual for a command. (Tap q to quit) |
| `sleep 10` | Wait 10 seconds before stopping. |
| `yes hello` | print `hello` until the program is killed. |
| `cmd &` | run `cmd` in the background. |
| `date` | Display the current date and time. |
| `date "+Y-m-d 00:00:00"` | set the date and time. |

### File navigation and editing

| | |
|---|---|
| `cd` | Change directory. |
| `ls` | List files in the current directory. |
| `ls -l` | List files with detailed information. |
| `ls -a` | List all files, including hidden ones. |
| `pwd` | Print the current working directory. |
| `open` | Open a file with the default application. |
| `mkdir` | Create a new directory. |
| `rm` | Remove a file. |
| `rm -rf` | Remove a directory and its contents. |
| `cp` | Copy a file or directory. |
| `mv` | Move or rename a file or directory. |
| `touch` | Create a new empty file. |
| `cat` | Display the contents of a file. |
| `more` | `cat` for big files. |
| `tail` | Display the last few lines of a file. |
| `tail -f` | Continuously display new lines added to a file. |
| `head` | Display the first few lines of a file. |
| `stat` | Display detailed information about a file. |

To search the command history press `Ctrl + r`.

### OS Commands

| | |
|---|---|
| `sudo` | Execute a command with superuser privileges. |
| `sudo -l` | List permitted commands. |
| `whoami` | Display the current user. |
| `who` | Display all logged-in users. |
| `id` | Display user ID and group ID information. |
| `id -u` | Display the user ID. |
| `id -g` | Display the group ID. |
| `nproc` | show the number of processing units (CPU cores). |
| `ps` | Display currently running processes. |
| `top` | Show the current processes running alongside the cpu and ram usage. |
| `kill` | Terminate a process by its PID. |
| `kill -9` | Forcefully terminate a process by its PID. |
| `kill -15` | terminate and clean a process by its PID. |
| `reboot` | Restart the system. |
| `poweroff` | Shut down the system. |
| `useradd` | Add a new user. |
| `userdel` | Delete a user. |
| `passwd` | Change a user's password. |
| `su - user` | Switch to another user. |
| `chown user file` | Change the owner of a file to `user`. |
| `chown user:group file` | Change the owner and group of a file. |
| `chmod flag file -u tom` | Change the permissions of `file` for user `tom`. |
| `chmod +x file` | Add execute permission to a file. |
| `chmod +r file` | Add read permission to a file. |
| `chmod +w file` | Add write permission to a file. |
| `chmod -x file` | Remove execute permission from a file. |
| `chmod -r file` | Remove read permission from a file. |
| `chmod -w file` | Remove write permission from a file. |

An alternative way of shutting down or reboot is to use `shutdown [OPTION] [TIME] [MESSAGE]` where `MESSAGE` shows a message to all users.

| **[TIME]** | |
|---|---|
| `now` | run shutdown instantly. |
| `+m` | run shutdown after `m` minutes. |
| `hh:mm` | run shutdown at the absolute time `hh:mm`. |
| **[OPTION]** | |
| `-P` | This is the default option. |
| `-H` | Halt |
| `-r` | reboot |
| `--no-wall` | shutdown without warning. |

## Daemons

Daemons are a type of process.

### Systemd

Systemd is the most widely used init system and service manager in modern Linux distributions. It is used in distributions such as Ubuntu, Debian, Fedora, and many others.

| | |
|---|---|
| `systemctl start service` | Start a service. |
| `systemctl stop service` | Stop a service. |
| `systemctl restart service` | Restart a service. |
| `systemctl status service` | Check the status of a service. |
| `systemctl enable service` | Enable a service to start on boot. |
| `systemctl disable service` | Disable a service from starting on boot. |
| `systemctl daemon-reload` | Reload the systemd manager configuration |
| `systemctl reset-failed service` | Reset the failed state of a service. |
| `journalctl -u service` | View logs for a specific service. |
| `journalctl -n 100 -u service` | View the last 100 log entries for a specific service. |
| `--no-pager` | Disable paging of log output. |

### OpenRC

OpenRC is an alternative init system and service manager used in some Linux distributions, such as Alpine Linux and Gentoo.

| | |
|---|---|
| `rc-service myService start` | Start a service. |
| `rc-service myService stop` | Stop a service. |
| `rc-service myService restart` | Restart a service. |
| `rc-service myService status` | Check the status of a service. |
| `rc-update add myService default` | Enable a service to start on boot. |
| `rc-update del myService default` | Disable a service from starting on boot. |

# Shell Scripting

## operators

| | |
|---|---|
| `>` | Redirect output to a file, overwriting the file if it exists. |
| `>>` | Redirect output to a file, appending to the file if it exists. |
| `<` | Redirect input from a file. |
| `\|` | Pipe the output of one command to another command. |
| `&&` | Execute the next command only if the previous command succeeded. |

## Basic Syntax

| | |
|---|---|
| `#!/bin/bash` | Shebang line to specify the script interpreter. |
| `echo` | Print text to the terminal. |
| `read` | Read input from the user. |
| `if` | Start of a conditional statement. |
| `elif` | Else if condition in a conditional statement. |
| `else` | Else condition in a conditional statement. |
| `fi` | End of a conditional statement. |
| `for` | Start of a for loop. |
| `while` | Start of a while loop. |
| `do` | Start of the loop body. |
| `done` | End of a loop. |
| `$0` | The name of the script. |
| `$1, $2, ...` | Positional parameters for script arguments. |
| `grep` | Search for a pattern in a file or input. |
| `exec` | Execute a command. |
| `exit` | Exit the script. |

## Networking Commands

The below commands requires the `net-tools` package to be installed on the system.

| | |
|---|---|
| `ifconfig` | Displays IP address. |
| `ping` | Returns packet loss between two the device and a destination. |
| `traceroute` | Traces the path packets take to a destination. |
| `nslookup` | Returns IP address information for a domain name. |
| `arp -a` | Displays the ARP table, which maps IP addresses to MAC addresses. |
| `arp myIPa` | Displays the ARP table for the specific IP address. |
| `curl` | Transfers data from or to a server, using various protocols. |
| `curl ifconfig.me` | Display machine's public IPv4 address. |

# SSH connection

| | |
|---|---|
| `ssh -i ~/.ssh/myKey user@ipv4` | Connect to a remote host via SSH. |
| `exit` | Exit the SSH session. |
| `Ctrl + D` | Exit the SSH session. |

# nginx

`nginx` is a software used to run a server on linux.

# Cron

Cron is a time-based job scheduler. It allows users to schedule tasks (commands or scripts) to run automatically at specified intervals.

## Basics

| | |
|---|---|
| `crontab -l` | List the current user's crontab. |
| `crontab -e` | Edit the current user's crontab. |
| `crontab -r` | Remove the current user's crontab. |
| `crontab filename` | Install a new crontab from a file. |
| `crond` | The cron daemon sometime also called `cron`. |

## Syntax

| | |
|---|---|
| `* * * * * command` | Run `command` every minute. |
| `0 * * * * command` | Run `command` at the start of every hour. |
| `0 0 * * * command` | Run `command` at midnight every day. |
| `0 0 * * 0 command` | Run `command` at midnight every Sunday. |
| `0 0 1 * * command` | Run `command` at midnight on the first day of every month |
| `0 0 1 1 * command` | Run `command` at midnight on January 1st every year. |

After the time and date fields, you can specify the command to be executed (note that cron dont use $PATH so you need to use absolute paths). Example:

```
0 0 * * 0 /usr/bin/python3 /home/user/myscript.py
0 5 * * * /sbin/reboot
```

This will run the Python script `myscript.py` at midnight every Sunday and reboot the system at 5 AM every day.

# package managers

| | |
|---|---|
| `apt` | Debian-based(include ubuntu) package manager. |
| `apk` | Alpine Linux package manager. |
| `yum` | Red Hat-based package manager. |
| `dnf` | Fedora-based package manager. |
| `pacman` | Arch Linux package manager. |
| `brew` | Alternative package manager for macOS and many Linux distributions. |

## apk

| | |
|---|---|
| `update` | Update repository indexes. |
| `upgrade` | Install upgrades available from repositories . |
| `list` | List all available packages. |
| `add pkg` | Install a package named `pkg`. |
| `del pkg` | Remove a package named `pkg`. |
| `search pkg` | Search for a package named `pkg`. |
| `info pkg` | Display detailed information about a package named `pkg`. |

## apt

| | |
|---|---|
| `update` | Update the package list. |
| `upgrade` | Upgrade all installed packages to their latest versions. |
| `list` | List all available packages. |
| `list --upgradable` | List all packages that can be upgraded. |
| `install pkg` | Install a package named `pkg`. |
| `remove pkg` | Remove a package named `pkg`. |
| `search pkg` | Search for a package named `pkg`. |

## brew

| | |
|---|---|
| `update` | Update the package list. |
| `upgrade` | Upgrade all installed packages to their latest versions. |
| `list` | List all available packages. |
| `install pkg` | Install a package named `pkg`. `--only-dependencies` self explenatory. |
| `uninstall pkg` | Remove a package named `pkg`. |
| `bundle install pkg` | Install a package named `pkg` and its dependencies. |

# Tips

## Render a website as an app

If a app is not available for linux but can be used on a web browser (eg: outlook, spotify, etc.). you can use the command `chromium-browser --app=siteUrl` (eg: `chromium-browser --app=https://outlook.office365.com/mail`). Create a file `~/.local/share/applications/outlook.desktop` with the following inside:

```
[Desktop Entry]
Name=Outlook
Comment=Outlook in Chromium
Exec=chromium-browser --app=https://outlook.office.com/mail/
Icon=internet-mail
Type=Application
Categories=Network;Email;
StartupWMClass=outlook.office.com__mail
```